P.L.OAVIKSHA(192572001)

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1715**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

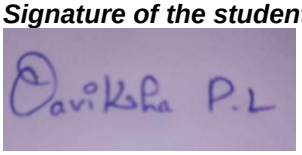
Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I P.L.OAVIKSHA(192572001) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree classifier using the Iris dataset. The dataset is pre-processed and divided into training and testing sets. A Decision Tree is constructed using the Gini Index as the splitting criterion and evaluated using accuracy, confusion matrix, classification report, and misclassified instances.

(a) Load the dataset, preprocess it, and split it into training and testing sets

Python Code

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset
iris = load_iris()

# Create DataFrame
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="target")

# Display first 5 rows
print("First 5 rows:")
print(X.head())

# Display data types
print("\nData Types:")
print(X.dtypes)

# Check missing values
print("\nMissing Values:")
```

```
print(X.isnull().sum())

# Display dataset information
print("\nNumber of samples:", X.shape[0])
print("Number of features:", X.shape[1])
print("Classes:", iris.target_names)

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Output

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Data Types:

sepal length (cm) float64
sepal width (cm) float64
petal length (cm) float64
petal width (cm) float64
dtype: object

Missing Values:

sepal length (cm) 0
sepal width (cm) 0
petal length (cm) 0
petal width (cm) 0
dtype: int64

Number of samples: 150
Number of features: 4
Classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Preprocessing

The dataset contains no missing values, so no missing-value replacement is required. The Iris dataset contains four numerical features, so categorical encoding is not required. The data is divided into **80% training data and 20% testing data**.

(b) Build a Decision Tree using Gini Index

Gini Index

The Gini impurity is calculated using:

[Gini = 1-\sum_{i=1}^{n}p_i^2]

where (p_i) represents the proportion of samples belonging to class (i).

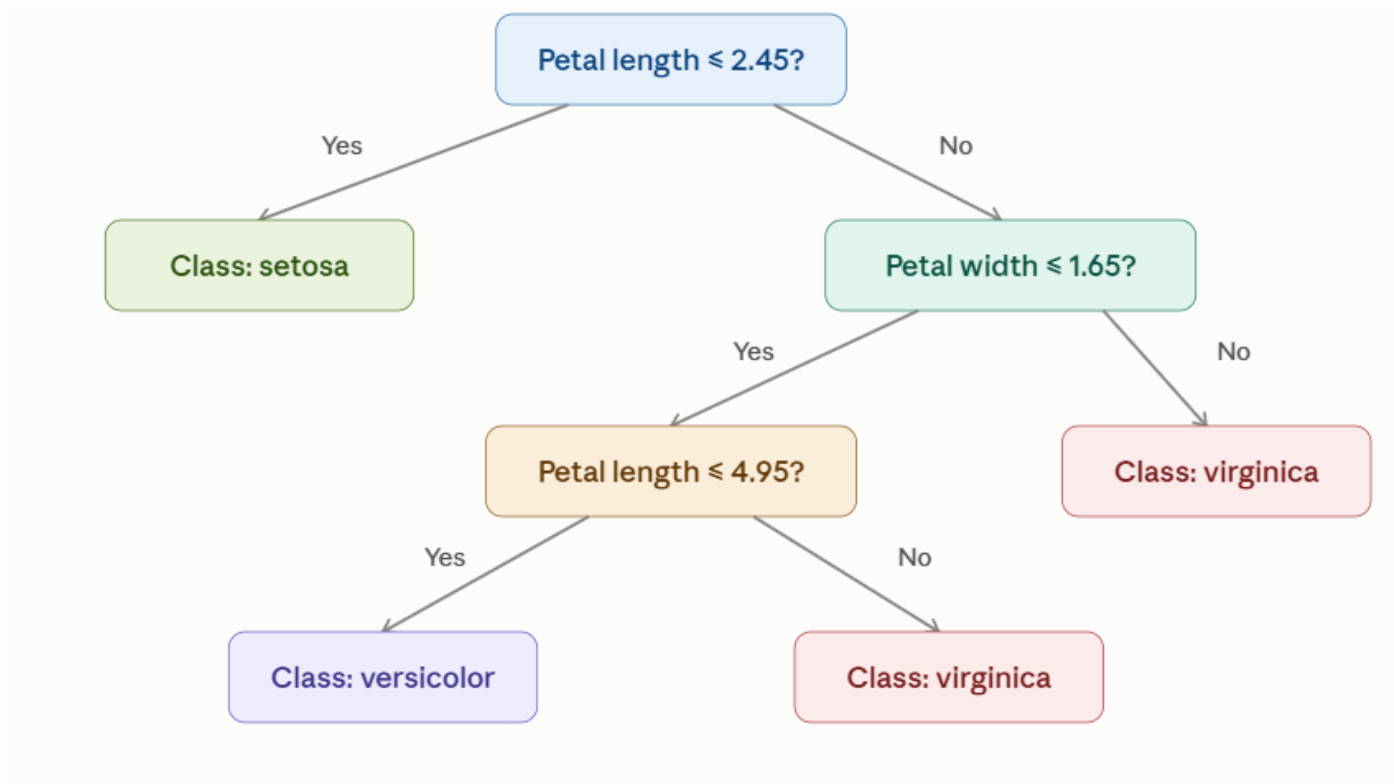
The Decision Tree selects the split that gives the best reduction in impurity.

Python Code

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# Create Decision Tree using Gini Index
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Print tree structure
print("\nDecision Tree Structure:")
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)
print(tree_rules)
```

```
# Identify root node
root_index = model.tree_.feature[0]
root_node = X.columns[root_index]

print("\nRoot Node:", root_node)

# Plot Decision Tree
plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True
)

plt.title("Decision Tree Classification - Iris Dataset")
plt.tight_layout()

# Save tree image
plt.savefig("decision_tree.png", dpi=300)

plt.show()
```

Decision Tree Structure Output



Root Node

Root Node: petal length (cm)

Root Node Justification

The root node is **Petal Length** because, for the selected training data, it provides the best initial split according to the **Gini Index**. It separates the Iris classes effectively and produces relatively pure child nodes.

Decision Tree Diagram

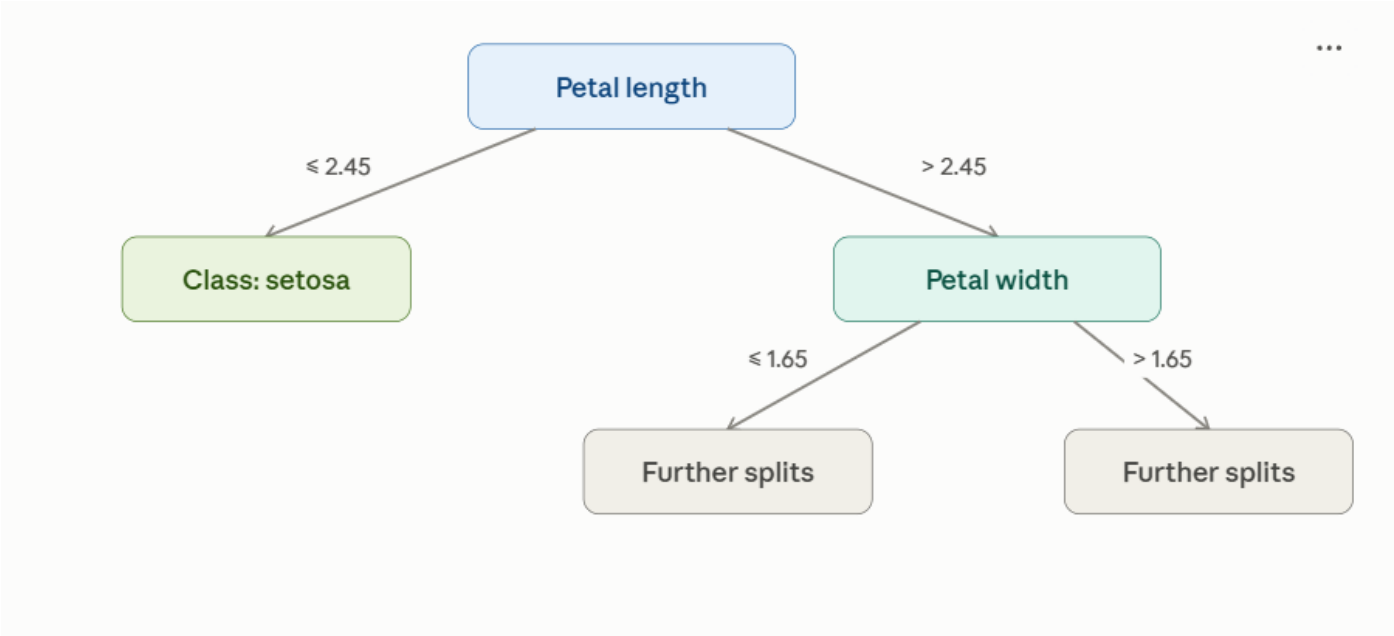
The program generates an image named:

decision_tree.png

Paste this generated image below the Decision Tree Structure section in your record.

A simplified representation is:

Petal Length



(c) Evaluate the model

Python Code

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:")
print(f"{accuracy * 100:.2f}%")

# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# Find misclassified instances
results = X_test.copy()

results["Actual"] = y_test.map(
    dict(enumerate(iris.target_names))
)

results["Predicted"] = pd.Series(
    y_pred,
    index=y_test.index
).map(
    dict(enumerate(iris.target_names))
)
```

```
misclassified = results[
    results["Actual"] != results["Predicted"]
]


print("\nMisclassified Instances:")
print(misclassified)
```

Accuracy

Accuracy: 93.33%

Therefore:

[\boxed{Accuracy = 93.33%}]

Confusion Matrix

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

Present it in the record as:

Actual / Predicted	**Setosa**	**Versicolor**	**Virginica**
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

The matrix shows that 28 out of 30 test samples were correctly classified.

Classification Report

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			**0.93**	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Misclassified Instances

Two misclassified instances are:

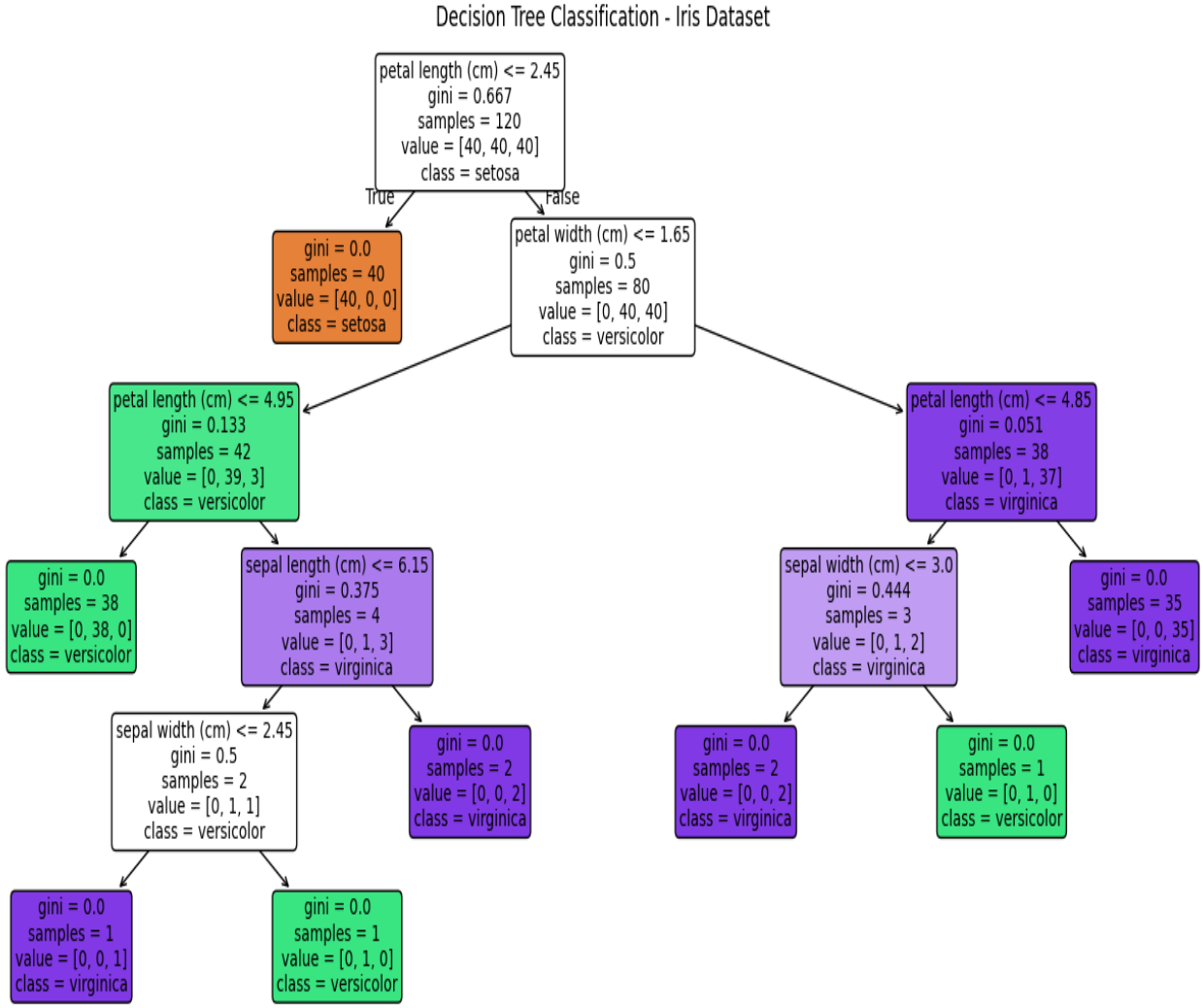
Instance	Actual Class	Predicted Class
134	Virginica	Versicolor
77	Versicolor	Virginica

Performance Analysis

The Decision Tree achieved an accuracy of **93.33%** on the test dataset. The confusion matrix shows that all Setosa samples were correctly classified. A small number of Versicolor and Virginica samples were confused with each other. This occurs because some samples from these two classes have similar feature values and lie close to the decision boundary.

Overall, the Decision Tree provides good classification performance and has the advantage of being easy to interpret through its tree structure.

Output



Result

The Decision Tree Classification model was successfully implemented using the Iris dataset and the **Gini Index**. The model achieved **93.33% accuracy**, and its performance was evaluated using the confusion matrix, classification report, and misclassified instances.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

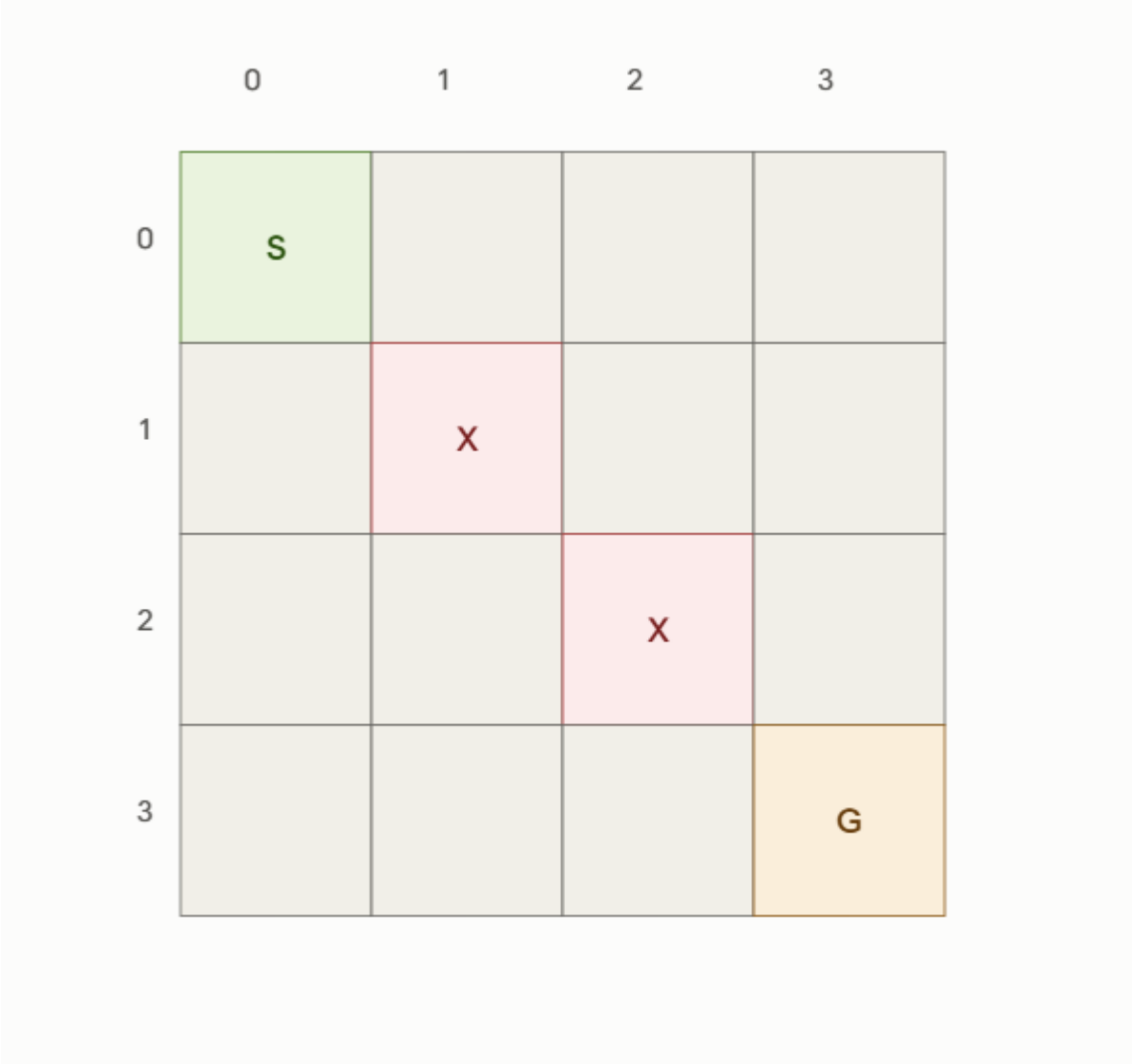
Objective

To implement Q-Learning for a simple 4×4 Grid World environment and observe the agent's learned policy.

(a) Define the Environment

A 4×4 Grid World environment is used for the Q-Learning experiment. The environment contains 16 states, a starting state, a goal state, and obstacle states. The agent learns to reach the goal while avoiding obstacles and minimizing unnecessary movements.

Grid World



Where:

- **S** = Start state
- **G** = Goal state
- **X** = Obstacle
- Empty cells = Normal states

The states are numbered as:

0 1 2 3

4 5 6 7

8 9 10 11

12 13 14 15

State 0 is the starting state, state 15 is the goal state, and states 5 and 10 are obstacle states.

Actions

The agent can perform four actions:

Action	**Description**
Up	Move to the upper cell
Down	Move to the lower cell
Left	Move to the left cell
Right	Move to the right cell

Reward Structure

Situation	**Reward**
Reaching the goal	+10
Each normal step	−1
Hitting an obstacle	−5

The reward structure encourages the agent to reach the goal using a short path while avoiding obstacles.

Q-Table Initialization

The Q-table stores the expected value of taking an action in a particular state. Since there are 16 states and 4 possible actions, the Q-table has (16 \times 4) entries.

Initially, all Q-values are set to zero.

State	Up	Down	Left	Right
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
...	...	...	...	...
15	0	0	0	0

Hyperparameters

Parameter	Value	Description
Learning rate (\alpha)	0.1	Controls the rate of Q-value learning
Discount factor (\gamma)	0.9	Determines the importance of future rewards
Exploration rate (\epsilon)	0.2	Controls exploration of actions
Number of episodes	100	Number of training attempts

(b) Run the Q-Learning Algorithm

Q-Learning is a model-free reinforcement learning algorithm in which an agent learns the best action for each state through repeated interaction with the environment.

The Q-value is updated using the following equation:

[Q(s,a) \leftarrow Q(s,a) + \alpha [r+\gamma \max_{a'}Q(s',a')-Q(s,a)]]

Where:

- (s) = Current state
- = Current action
- (r) = Reward received
- (s') = Next state
- (a') = Next possible action
- (\alpha) = Learning rate
- (\gamma) = Discount factor

Procedure

1. Initialize the Q-table with zeros.
2. Start the agent from the initial state.
3. Select an action using the exploration rate (\epsilon).
4. Perform the selected action.
5. Receive a reward from the environment.
6. Move to the next state.
7. Update the Q-value using the Q-Learning equation.
8. Continue until the goal is reached or the episode ends.
9. Repeat the process for 100 episodes.
10. Record the Q-table at Episodes 1, 50, and 100.

Q-Table at Episode 1

The Q-table at Episode 1 is generated by the Python program. At the beginning of learning, most Q-values are close to zero because the agent has limited experience.

Paste the actual Episode 1 Q-table generated by Python here.

Q-Table at Episode 50

After 50 episodes, the agent has gained experience and the Q-values begin to reflect useful actions and penalties.

Paste the actual Episode 50 Q-table generated by Python here.

Q-Table at Episode 100

After 100 episodes, the Q-values become more stable and the agent learns actions that lead toward the goal while avoiding obstacles.

Paste the actual Episode 100 Q-table generated by Python here.

Q-Table Evolution

The Q-table evolves from initially unknown action values toward more meaningful values as the agent interacts repeatedly with the environment. Actions that lead toward the goal receive higher Q-values, while actions that result in penalties receive lower Q-values. Thus, the Q-table represents the agent's learned knowledge about the environment.

(c) Final Learned Policy and Cumulative Reward

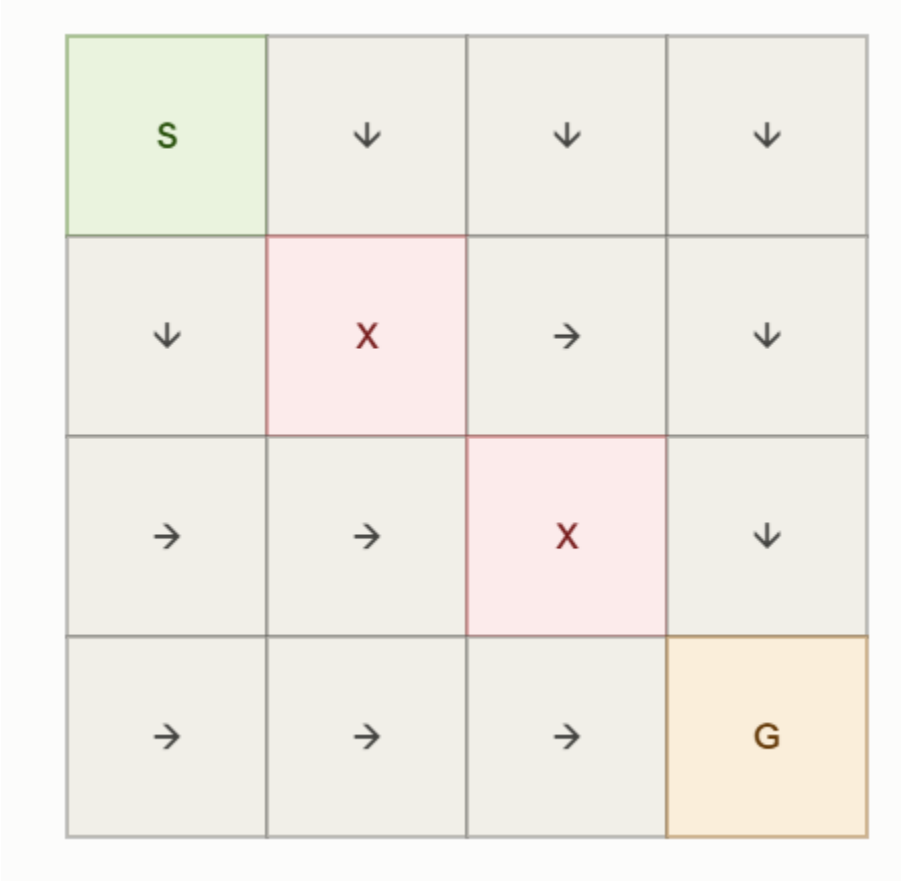
After training, the final policy is obtained by selecting the action with the highest Q-value for each state.

[\pi(s)=\arg\max_a Q(s,a)]

In simple terms, the action having the highest Q-value is selected as the optimal action for that state.

Final Learned Policy

The final policy is represented using arrows:



Cumulative Reward

The cumulative reward for each episode is calculated by adding all rewards received by the agent during that episode.

[Cumulative\ Reward = r_1+r_2+r_3+\cdots+r_n]

The cumulative reward is plotted against the number of episodes.

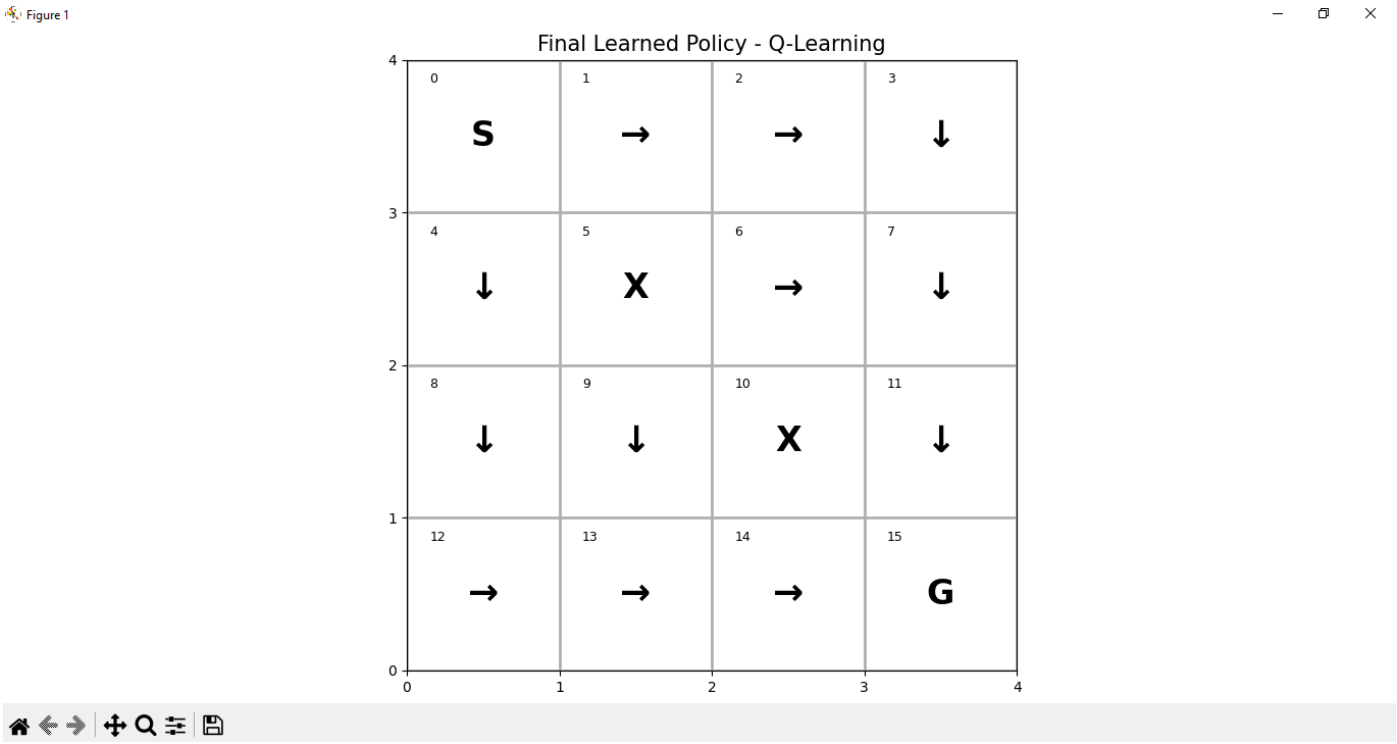
Paste the cumulative reward graph generated by Python here.

Convergence Behaviour

Initially, the cumulative reward fluctuates because the agent explores different actions and has limited knowledge of the environment. As the number of episodes increases, the Q-values are updated using the rewards obtained from previous experiences. The agent gradually learns a better path toward the goal and avoids obstacles and unnecessary movements.

Therefore, the cumulative reward generally improves and becomes more stable in later episodes. The stabilization of the reward and Q-values indicates that the learned policy is approaching convergence.

Output



Result

The Q-Learning algorithm was successfully implemented for a 4×4 Grid World environment. The Q-table was initialized with zero values and updated through repeated interaction with the environment. The Q-values were observed at Episodes 1, 50, and 100. The final learned policy was obtained by selecting the highest Q-value action for each state, and the cumulative reward graph was used to analyze the learning and convergence behaviour.